



Universidad Nacional de La Matanza
Departamento de Ingeniería e Investigaciones Tecnológicas

Programación Concurrente (3658)

Curso: 01 3900

Grupo M2

Informe académico

Métricas IA

Análisis de código generado con herramientas Vibecoding

Fecha de Presentación: 27/05/2026

Integrantes

DNI	Nombre	Apellido	Email
41246810	Kevin Nahuel	Arias	kearias@alumno.unlam.edu.ar
40955907	Marcos	Costanzo	mcostanzo@alumno.unlam.edu.ar
40015557	Cristian	Rios	crrios@alumno.unlam.edu.ar
43744593	Agustín	Panigazzi	apanigazzi@alumno.unlam.edu.ar

Introducción	2
Objetivo del trabajo.....	2
Marco teórico: Programación Concurrente	2
Marco teórico: Métricas de Calidad de Software.....	2
Desarrollo	2
Prompt para el Sistema de venta de entradas.....	2
Análisis del código generado.....	2
Fortalezas	2
Características	2
Iteraciones y problemas encontrados.....	2
Eficiencia	2
Claridad	2
Adecuación a la concurrencia.....	2
Resultados	2
Métricas	2
Complejidad ciclomática	2
Mantenibilidad.....	2
Rendimiento (cantidad de procesos y threads utilizados).....	2
Uso de CPU y memoria.....	2
Cantidad de tokens utilizados.....	2
Cantidad de refinamientos	2
Grado de correctitud del código en forma concurrente (sincronización adecuada, deadlocks, race condition, exclusión mutua)	2
Conclusiones	2
Referencias	2

Introducción

Objetivo del trabajo

El objetivo de este trabajo es evaluar la viabilidad y efectividad de utilizar herramientas de Inteligencia Artificial generativa para el desarrollo de sistemas de programación concurrente. Como caso de estudio, se desarrolló un sistema de venta de entradas para recitales que debe soportar cientos de usuarios simultáneos intentando reservar los mismos asientos.

El trabajo también propone la aplicación de métricas de calidad de software para evaluar cuantitativamente el código generado, permitiendo medir no solo si funciona, sino qué tan bien funciona en términos de eficiencia, mantenibilidad y correctitud concurrente.

Marco teórico: Programación Concurrente

La programación concurrente es el área de la informática que estudia cómo múltiples tareas pueden ejecutarse simultáneamente, compartiendo recursos limitados. Es fundamental en sistemas modernos: desde servidores web que atienden miles de solicitudes por segundo, hasta aplicaciones móviles que ejecutan operaciones en segundo plano sin bloquear la interfaz.

El principal desafío de la concurrencia es evitar que dos procesos accedan simultáneamente al mismo recurso de forma conflictiva. Cuando esto ocurre sin control, se genera una **race condition**: el resultado depende del orden impredecible en que se ejecutan las operaciones. En un sistema de venta de entradas, una race condition significaría que dos usuarios compren el mismo asiento.

Para prevenir esto existen mecanismos como:

- **Locks (candados)**: bloquean un recurso para que solo un proceso pueda accederlo
- **Semáforos**: controlan el acceso contando cuántos procesos pueden usar un recurso
- **Operaciones atómicas**: garantizan que una operación ocurra completamente sin interrupciones
- **Transacciones**: agrupan múltiples operaciones que deben ejecutarse juntas o no ejecutarse

En nuestro caso, elegimos delegar la sincronización a operaciones atómicas en la base de datos, aprovechando que PostgreSQL garantiza atomicidad en sus consultas SQL.

Marco teórico: Métricas de Calidad de Software

Las métricas de software son mediciones cuantitativas que ayudan a evaluar la calidad del código. No basta con que un programa "funcione"; también importa cuán mantenible sea, cuán eficiente, y cuán propenso está a errores.

Las métricas aplicadas en este trabajo son:

- **Complejidad ciclomática:** mide cuántos caminos diferentes puede tomar un programa. Código con alta complejidad es más difícil de entender y probar. Se calcula sumando los puntos de decisión (if, while, for, etc.) en una función.
- **Mantenibilidad:** evalúa cuán fácil es modificar el código. Un sistema bien organizado, con funciones pequeñas y responsabilidades claras, es más mantenible.
- **Rendimiento:** mide throughput (requests por segundo) y latencia (tiempo de respuesta). Es crítico en sistemas con demanda concurrente.
- **Uso de recursos:** CPU y memoria consumidas. Un sistema bien diseñado usa eficientemente los recursos disponibles.
- **Correctitud concurrente:** verifica que el sistema maneje correctamente los accesos simultáneos. ¿Hay race conditions? ¿Deadlocks? ¿La sincronización es adecuada?

Desarrollo

Prompt para el Sistema de venta de entradas

Descripción general

Desarrollar un "Sistema de Venta de Entradas" para Recitales. Para el mismo se deben utilizar conceptos de Programación concurrente (threads, procesos, sincronización, locks, colas, etc.). No es necesario que se usen todos los conceptos, solo los necesarios para esta solución. En esta solución no se deben utilizar sistemas externos como por ejemplo AWS, sino que los mecanismos utilizados deben ser propio del Sistema Operativo.

Alcance del sistema

El sistema de venta de entradas tiene que permitir a los usuarios ver todos los asientos disponibles para un determinado recital, para poder seleccionarlos y reservarlos. Hay que tomar en cuenta que los asientos pueden estar DISPONIBLES, RESERVADOS TEMPORALMENTE o VENDIDO.

El manejo de la concurrencia en este sistema es crítico y vital. Puede haber cientos o miles de usuarios intentando reservar entradas al mismo tiempo y pueden querer reservar los mismos asientos.

Cuando un usuario selecciona un asiento, el sistema se lo reserva por 5 MINUTOS (parámetro configurable), momento en que pasa al estado RESERVADO TEMPORALMENTE. Si pasa el tiempo y no se confirma (vende) la reserva, el sistema “libera” el asiento disponibilizándolo para otros usuarios. Mientras esté temporalmente reservado, a todos los usuarios que lo estaban visualizando como disponible tienen que verlo marcado con el nuevo estado, sin necesidad de actualizar la página. Lo mismo cuando se libera el asiento.

Lenguaje de Programación

Python 3.14

Plataforma dónde se ejecutará

GNU Linux

Agente de IA utilizado para la implementación

Claude Code (Haiku 4.5, Sonnet 4.6 y Opus 4.7)

Requisitos específicos

- El sistema debe ser frontend con un backend al cual realizar consultas. Es el backend quien debe tener toda la lógica de negocio.
- El sistema debe funcionar todo en un mismo servidor.
- El sistema debe utilizar una base de datos PostgreSQL versión 18.
- El sistema debe tener un login básico y autenticarse contra la DB, con información mínima de cada usuario.
- El sistema tiene que manejar la concurrencia para la selección de asientos mediante la base de datos. Ejemplo: una query que actualice el estado del asiento únicamente si ese asiento está disponible (WHERE = 'available'). De esta manera, ya que las queries en la base de datos me garantizan atomicidad, estaría resolviendo el problema de la race condition.
- El sistema debe soportar demanda concurrente, considerar de cientos a miles de usuarios en el mismo momento, considerando también optimización de memoria e hilos.
- El sistema debe evitar deadlocks y problemas de concurrencia (acceso simultáneo al mismo recurso cuando ya está siendo tomado por otro recurso).
- El sistema debe utilizar conceptos como Threads, Procesos, Sincronización, Locks, Colas, etc. (no es necesario usar todos, solo los que mejor se adecuen a la solución) garantizando un correcto uso de recursos.
- El sistema debe gestionar lo que es la selección de asientos únicamente y poder luego continuar con el pago.
- Los clientes deben poder seleccionar uno o más asientos al momento de comprar la entrada.

- El acceso será concurrente, es decir, puede ocurrir que dos personas quieran seleccionar el mismo asiento al mismo tiempo. Solo uno debe poder seleccionarlo, al otro se le debe mostrar un mensaje en el frontend indicándole que fue reservado.
- El sistema debe permitir sacar entradas en únicamente 3 secciones: Campo, Platea y Platea VIP.
- El sistema debe validar la disponibilidad de asientos seleccionados al momento de iniciar el proceso de compra.
- El sistema debe “reservar” temporalmente los asientos seleccionados por el cliente hasta que pase el tiempo de compra o se cancele la compra (Botón Cancelar).
- Ya que el sistema debe estar enfocado en la resolución de la concurrencia en la reserva y venta de entradas, el alta/baja/modificación de recitales no es prioritario para este sistema. Por lo tanto, se deberá la carga de todos los asientos que disponibiliza un recital por archivos JSON, siendo el sistema el responsable de parsear y manejar los asientos a partir de esos datos. Esto con el fin de simplificar el manejo de los recitales. Se requiere crear varios archivos de prueba, para realizar la carga. Los usuarios, en el front, una vez logueados deben seleccionar el recital a asistir de un listado para visualizar los asientos disponibles.
- En el frontend, los colores de los asientos disponibles deben ser de color verde y se debe actualizar en tiempo real los asientos reservados en color rojo.
- El sistema debe escribir un log cada vez que se detecte una race condition. Ejemplo: el usuario A quiso reservar el asiento C1 al mismo tiempo que el usuario B. En ese caso la base de datos maneja esa race condition ya que sus sentencias son atómicas y si un usuario B obtiene que se modificaron 0 filas, eso significaría que el asiento está reservado por el usuario A en este caso. Esto debe ser informado en un log, avisando que se detectó la condición de carrera y que solo uno de los dos usuarios obtuvo el asiento.
- Se debe generar un batch o casos de prueba para evaluar correctamente los casos de race condition.

Restricciones

- El pago debe simularse con stubs, permitiendo avanzar en la reserva del asiento sin realizar ninguna operación real contra algún sistema externo, de forma de poder implementar el servicio de pagos a futuro.
- La selección de asientos en la interfaz (frontend) no garantiza su disponibilidad.
- Los asientos seleccionados por el cliente se liberan una vez que al cliente le pasan los 10 minutos de tiempo de compra.

- Al momento de que un cliente selecciona un asiento, el mismo debe actualizarse para todos los demás en la interfaz (frontend), quedándose así bloqueada para todos los demás.
- Si un cliente A seleccionó un asiento y al mismo tiempo (o 1 segundo después) otro cliente B también lo seleccionó sin haberse actualizado la interfaz (frontend), el cliente B debería ver un mensaje de “ese asiento ya fue seleccionado” por más que el asiento no esté marcado en el frontend como reservado.
- Los recursos del backend/lógica de negocio deben utilizar mecanismos de sistema operativo como Threads, Procesos, Sincronización, Locks, Colas, etc.

Implementación por Vibe Coding

A partir del prompt, el agente de Claude Code comenzó a realizar todo el proyecto, plantear la arquitectura, realizar los casos de prueba y la configuración de Docker.

El agente realizó la siguiente arquitectura:

Backend:

- Punto de entrada principal en main.py
- Parámetros de configuración del sistema en config.py
- Pool de conexiones PostgreSQL en database.py
- Autenticación con JWT
- Modelos (estructura de requests y responses)
- Lógica de negocio para asignación de asientos en seat_service.py
- Manager de WebSockets, para que los usuarios vean actualizaciones en tiempo real de la disponibilidad de usuarios sin tener que refrescar la página en ws_manager.py
- Thread de limpieza de reservas
- Un cleaner de asientos temporalmente reservados que ya expiraron (y vuelven a estar disponibles) en cleaner.py
- Cargador de recitales desde JSON en concert_loader.py
- Logger de race conditions race_logger.py
- Stub de procesamiento de pagos payment_stub.py para implementación a futuro

El agente usó Swagger para la documentación de los endpoints, y su visualización simple en un endpoint de /docs:

Sistema de Venta de Entradas para Recitales

1.0.0 OAS 3.1

openapi.json

Authorize

default

POST	/api/auth/login	Login	
GET	/api/auth/me	Get Me	 
GET	/api/concerts	List Concerts	
GET	/api/concerts/{concert_id}	Get Concert	
GET	/api/concerts/{concert_id}/seats	List Seats	
GET	/api/concerts/{concert_id}/my-seats	My Reserved Seats	
POST	/api/seats/reserve	Reserve	
POST	/api/seats/release	Release	
POST	/api/payment/process	Process	
GET	/api/admin/race-conditions	Race Conditions	
GET	/	Root	
GET	/concerts	Concerts Page	
GET	/seats	Seats Page	
GET	/payment	Payment Page	

Como se puede ver, el agente implementó endpoints de autenticación y login, de gestión de conciertos y reserva de asientos, un endpoint de administrador para ver los logs de race conditions ocurridos, y view endpoints para cargar las vistas del sistema.

Para el frontend, implementó vistas muy simples en HTML y CSS con scripts de JS embebidos en bloques `<script>`, y funciones utilitarias en un `app.js`. Se priorizó minimizar la complejidad, según los requerimientos que le dimos al agente, por lo que no implementó React o Angular o algún framework más complejo de front.



Sistema de Venta de Entradas

Bienvenido

Usuario

Contraseña

Ingresar

Usuario de prueba: **test_user**

Contraseña: **test123**

Jazz Fusion - Una Noche de Improvisación

Fecha: 22/6/2026

Lugar: Teatro Colón, Buenos Aires

ESCENARIO

CAMPO

A	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
B	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
C	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
D	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
E	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
F	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
G	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
H	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

PLATEA

A	1	2	3	4	5	6	7	8	9	10	11	12
B	1	2	3	4	5	6	7	8	9	10	11	12
C	1	2	3	4	5	6	7	8	9	10	11	12
D	1	2	3	4	5	6	7	8	9	10	11	12
E	1	2	3	4	5	6	7	8	9	10	11	12
F	1	2	3	4	5	6	7	8	9	10	11	12

PLATEA VIP

A	1	2	3	4	5	6	7	8
B	1	2	3	4	5	6	7	8
C	1	2	3	4	5	6	7	8

Mi Selección

CAMPO A10	\$3.500,00
CAMPO A11	\$3.500,00

Subtotal:	\$7.000,00
Total:	\$7.000,00

Reservar Asientos

Cancelar

 Entradas

test_user
Cerrar Sesión

Confirmación de Compra

Resumen del Pedido

Recital: Jazz Fusion - Una Noche de Improvisación

Fecha: 22/6/2026

Lugar: Teatro Colón, Buenos Aires

Asientos:

CAMPO A10	\$3.500,00
CAMPO A11	\$3.500,00

Subtotal: \$7.000,00

Impuestos (0%): \$0.00

Total: \$7.000,00

Método de Pago

☒ Tarjeta de Crédito
☐ Tarjeta de Débito
☐ Transferencia Bancaria

Confirmar Compra
Volver

El modelo de datos es muy simple, teniendo solamente las tablas de users, concerts, seats y log de race conditions:

Tabla “Users”:

columna	tipo	detalle
id	SERIAL	PK
username	VARCHAR(50)	único
password_hash	VARCHAR(255)	
email	VARCHAR(100)	
full_name	VARCHAR(100)	

created_at	TIMESTAMPTZ	default NOW()
------------	-------------	---------------

Tabla “Concerts”:

columna	tipo	detalle
id	SERIAL	PK
name	VARCHAR(200)	
artist	VARCHAR(200)	
event_date	TIMESTAMPTZ	
venue	VARCHAR(200)	
description	TEXT	
json_file	VARCHAR(255)	único, evita cargar el mismo archivo dos veces
is_active	BOOLEAN	default TRUE
created_at	TIMESTAMPTZ	default NOW()

Tabla “Seats”:

columna	tipo	detalle
id	SERIAL	PK
concert_id	INTEGER	FK → concerts
section	VARCHAR(20)	campo / platea / platea_vip
row_label	VARCHAR(10)	
seat_number	VARCHAR(10)	
status	VARCHAR(20)	available / reserved / sold
price	NUMERIC(10,2)	
reserved_by	INTEGER	FK → users, NULL si no está reservado
reserved_at	TIMESTAMPTZ	
reservation_expires_at	TIMESTAMPTZ	
sold_at	TIMESTAMPTZ	
sold_to	INTEGER	FK → users

Tabla “race condition log”:

columna	tipo	detalle
id	SERIAL	PK
seat_id	INTEGER	FK → seats
loser_user_id	INTEGER	FK → users
loser_username	VARCHAR(50)	
thread_name	VARCHAR(100)	
occurred_at	TIMESTAMPTZ	default NOW()

El agente también creó batchs de prueba acorde para probar la concurrencia en casos border:

- load-test → tests/concurrent_test.py

Simula 10 usuarios peleando por el mismo asiento y comprueba que solo uno lo obtenga.

- perf-test → tests/performance_test.py

Manda 100 reservas concurrentes que se distribuyen entre 10 usuarios sobre asientos distintos y calcula métricas como requests/s, latencia promedio y del percentil 95.

Análisis del código generado

El código que implementó el agente Sí cumple con el objetivo esperado. Generó un proyecto con foco en el problema de la concurrencia al reservar asientos por múltiples usuarios al mismo tiempo. No generó un frontend muy complejo, ni servicios sobrecargados con lógica innecesaria, sino que implementó el servicio de reserva de forma robusta, así como crear archivos de pruebas de carga y concurrencia.

Hay que tomar en cuenta que, al ser para fines académicos, se tomaron algunas consideraciones:

- El usuario y contraseña de prueba para login está hardcodeado (las credenciales están visibles en la configuración).
- La key de JWT también está pre-seteada en la configuración.

Fortalezas

El proyecto implementado separa frontend, backend, base de datos y pruebas de concurrencia. La lógica de negocio se centra en el backend, evitando lógica innecesaria en el frontend para decidir si el asiento está realmente disponible. La arquitectura está bien planteada, cada servicio tiene una responsabilidad bien definida (el cleaner, el stub de payments, el servicio de reserva de asientos, etc.). El uso de un agente de IA favoreció mucho la construcción del proyecto siguiendo lineamientos conocidos y buenas prácticas del desarrollo del software.

La principal fortaleza concurrente es el uso de PostgreSQL como mecanismo de sincronización para la reserva de asientos. La operación crítica se implementa con un UPDATE condicional, y solo modifica el asiento si su estado actual es *available*. De esta manera, ante dos solicitudes simultáneas por el mismo asiento, la base de datos garantiza atomicidad y solo una logra reservarlo.

También se incorporan WebSockets para notificar cambios de estado en tiempo real, un proceso de limpieza de reservas vencidas ejecutado en segundo plano y un registro de logs de condiciones de carrera detectadas. Estos elementos muestran que la solución contempla tanto la prevención como la observabilidad de problemas concurrentes.

Como fortaleza adicional, se incorporó Docker Compose para levantar el backend y la DB de PostgreSQL. Esto facilita también la ejecución de las pruebas y el obtener métricas de rendimiento, evitando depender de configuraciones manuales específicas en cada máquina.

Características

- JWT con expiración configurable (default 120 min)
- PostgreSQL 18

- Pool de conexiones con mínimo 5, máximo 100 conexiones, timeout configurable de 30s
- Seed de datos
- Auditoría de race conditions — cada colisión queda registrada en `race_condition_log` con usuario, asiento, thread y timestamp
- Rollback automático ante cualquier fallo dentro de una transacción
- WebSocket por recital — los cambios de estado de asientos (reservado, liberado, vendido) se broadcastean a todos los clientes conectados en tiempo real
- Máquina de estados por asiento: `available` → `reserved` → `sold` (o `reserved` → `available`)
- Timeout de reserva configurable (default 5 min) — si el usuario no paga, el asiento se libera automáticamente
- Cleaner daemon — hilo de fondo que expira reservas vencidas cada 30 segundos (configurable), devolviendo asientos al pool y notificando por WebSocket
- Swagger UI en `/docs` — documentación de los endpoints
- Docker Compose
- Logging estructurado con timestamps a stdout en `/logs`
- Test de concurrencia (`concurrent_test.py`) — 10 usuarios simultáneos sobre el mismo asiento, verifica que solo 1 gane
- Test de rendimiento (`performance_test.py`) — throughput, latencia promedio, mínima, máxima y p95 bajo carga configurable
- Perfiles Docker — los tests corren como servicios aislados contra la misma red del stack

Iteraciones y problemas encontrados

El código generado inicialmente no fue perfecto. Al intentar ejecutarlo por primera vez, el proyecto no arrancaba: las dependencias estaban mal especificadas, el código hacía referencia a una librería vieja de conexión a PostgreSQL (`psycopg2`) pero usaba la API de su versión nueva (`psycopg3`), que son incompatibles entre sí. Además, el archivo de requisitos listaba versiones que no existían o que tenían conflictos entre ellas, y el README indicaba que requería Python 3.15, una versión que aún no está disponible. Fue necesario reescribir la lista de dependencias con versiones concretas y compatibles antes de poder levantar el servidor por primera vez.

Una vez que el servidor arrancó, se encontraron problemas en la lógica de concurrencia. Había dos bugs importantes. El primero era que cuando un usuario intentaba confirmar la compra de varios asientos a la vez, la consulta a la base de datos no verificaba que los asientos pertenecieran al recital correcto ni que la reserva no hubiera expirado. Esto significaba que un usuario podía comprar asientos de otro recital o asientos cuyo tiempo de reserva ya había vencido. El segundo bug era un potencial deadlock: si dos usuarios intentaban reservar los mismos asientos pero en distinto orden (uno pedía A y B, el otro pedía B y A), los dos procesos podían quedarse esperándose mutuamente. La solución fue ordenar siempre los IDs antes de procesarlos, forzando que todos sigan el mismo camino.

El test de concurrencia también tenía un error de diseño importante: todos los usuarios simulados estaban usando el mismo token de autenticación. Eso significa que para la base de datos no eran diez usuarios distintos peleando por un asiento, sino el mismo usuario haciendo diez pedidos. La prueba pasaba, pero no estaba probando lo que creía estar probando. Se corrigió autenticando a cada usuario por separado y manteniendo sus tokens individuales.

Al agregar el entorno Docker aparecieron dos problemas. El primero fue que los hashes de contraseñas precargados en la base de datos eran incorrectos — el modelo generó cadenas que no correspondían a las contraseñas reales, así que ningún usuario podía iniciar sesión. Hubo que regenerar todos los hashes correctamente. El segundo fue un bug en el manejo del token de autenticación: el estándar JWT define que cierto campo identificador debe ser siempre un texto, pero el código lo estaba guardando como número. Dentro de Docker, con una versión específica de la librería JWT, eso hacía fallar silenciosamente la autenticación de todos los usuarios.

Luego, con un análisis más cuidadoso del código se pudo ver que se usaba una librería de seguridad con vulnerabilidades conocidas y sin mantenimiento activo, no había ningún límite en los intentos de login (lo que permite ataques de fuerza bruta), y por cada request autenticado el sistema hacía una consulta innecesaria a la base de datos que no aportaba nada que no estuviera ya en el token. Este último punto tenía un efecto directo y medible: al eliminarlo, el throughput del sistema casi se duplicó.

Eficiencia

Desde el punto de vista de eficiencia, la solución evita implementar locks “manuales” prolongados en la aplicación. La exclusión mutua se delega a operaciones atómicas de PostgreSQL, lo que reduce la complejidad del backend y permite que las transacciones sean breves.

El uso de un pool de conexiones favorece la atención de solicitudes concurrentes sin abrir una conexión nueva por cada request. Además, la limpieza de reservas vencidas se ejecuta mediante un hilo de fondo, lo que permite liberar asientos expirados sin bloquear el flujo principal de atención HTTP.

El uso de WebSockets también mejora la eficiencia del frontend, porque evita que los clientes tengan que consultar constantemente el servidor para detectar cambios de estado en los asientos.

En la prueba de rendimiento realizada con 200 solicitudes y concurrencia 50, el sistema completó la ejecución en 0,601 segundos, con un throughput aproximado de 332,68 requests por segundo y 0 fallas. La latencia promedio fue de 0,1417 segundos y la latencia p95 de 0,2259 segundos. Estos resultados muestran un comportamiento eficiente para el escenario probado.

Claridad

La estructura del código es comprensible y está organizada por responsabilidades. La reserva, liberación y confirmación de asientos se encuentran concentradas en `seat_service.py`; la limpieza automática en `cleaner.py`; la administración de WebSockets en `ws_manager.py`; y el registro de condiciones de carrera en `race_logger.py`.

Esta separación facilita identificar los puntos críticos de concurrencia y permite analizar el sistema por módulos. Sin embargo, algunas funciones centrales, especialmente la reserva de asientos, concentran varias tareas: validación, actualización atómica, rollback, logging y notificación. En futuras iteraciones podría mejorarse la claridad separando parte de esas responsabilidades.

La incorporación de un documento específico para Docker también mejora la claridad operativa del proyecto, ya que describe cómo levantar el sistema, ejecutar pruebas y registrar métricas de rendimiento.

Adecuación a la concurrencia

La solución es adecuada para el problema planteado porque identifica correctamente el recurso compartido principal: los asientos. El sistema permite que múltiples usuarios intenten reservar al mismo tiempo, pero garantiza que un asiento solo pueda pasar de available a reserved una vez.

La estrategia utilizada controla las race conditions mediante la atomicidad de la base de datos. Cuando una solicitud concurrente no logra actualizar el asiento porque otro usuario lo reservó antes, el sistema detecta el conflicto, registra el evento y devuelve un mensaje al usuario.

Esta adecuación fue validada mediante un batch donde 10 usuarios intentaron reservar simultáneamente el mismo asiento. El resultado esperado se cumplió: solo una reserva fue exitosa y las otras nueve solicitudes recibieron un conflicto controlado. Esto confirma que la operación atómica sobre la base de datos resuelve correctamente la race condition principal del sistema.

También se emplean mecanismos propios de programación concurrente, como `threading.Thread`, `threading.Event`, `threading.Lock`, `asyncio` y `WebSockets`. Estos mecanismos son coherentes con el enunciado, ya que permiten resolver tareas de fondo, sincronizar estructuras compartidas y comunicar cambios de estado en tiempo real.

Resultados

Métricas

Complejidad ciclomática

La función con mayor complejidad ciclomática identificada es la encargada de reservar asientos (`reserve_seats`), en el módulo de servicio de asientos del backend, con una complejidad de 6. Para calcularlo se parte del valor base 1 de toda función y se suman los puntos de decisión reales:

- 1 base
- +1 ciclo que recorre los asientos solicitados
- +1 verificación de existencia del asiento en el camino de error
- +1 evaluación del resultado del intento de reserva atómica
- +1 manejo de excepciones inesperadas (`except`)
- +1 ciclo final que notifica a los clientes conectados por `WebSocket`

Complejidad ciclomática = 6

Esta complejidad es moderada y justificada: la función concentra la lógica más crítica del sistema. Primero intenta realizar la reserva directamente con una operación atómica en la base de datos — si nadie más llegó antes, el asiento queda reservado. Si la operación no tuvo efecto, significa que otro usuario se adelantó, y ahí se hace una consulta adicional solo para construir el mensaje de error y registrar el evento. Ante cualquier fallo inesperado, se deshacen todos los cambios de la transacción. Finalmente, cada asiento reservado con éxito dispara una notificación en tiempo real a todos los usuarios que estén viendo el mapa de ese recital. Cada uno de esos pasos genera una bifurcación en el flujo, lo que explica el valor obtenido.

Mantenibilidad

La mantenibilidad se evaluó observando la organización del código, la separación de responsabilidades y la facilidad para ubicar los componentes principales del sistema. El proyecto presenta una estructura clara, con módulos diferenciados para autenticación, acceso a base de datos, lógica de asientos, WebSockets, limpieza de reservas expiradas, carga de recitales y logging.

Esta separación favorece la comprensión del sistema y permite identificar con facilidad dónde se encuentra la lógica crítica de concurrencia. Por ejemplo, la reserva y confirmación de asientos se concentran en `seat_service.py`, mientras que la limpieza automática se encuentra en `cleaner.py` y la gestión de actualizaciones en tiempo real en `ws_manager.py`.

Como aspecto mejorable, algunas funciones centrales concentran varias tareas al mismo tiempo, como validación, actualización en base de datos, rollback, logging y notificación a clientes. Si el sistema creciera, sería conveniente separar parte de esa lógica en funciones auxiliares para mejorar la legibilidad y facilitar las pruebas.

La incorporación de Docker Compose y de una guía de ejecución también mejora la mantenibilidad operativa, ya que facilita reproducir el entorno de ejecución y repetir las pruebas de rendimiento sin configuraciones manuales complejas.

Rendimiento (cantidad de procesos y threads utilizados)

El rendimiento se analizó ejecutando el sistema completo mediante Docker Compose. La configuración utilizada levantó un contenedor para PostgreSQL 18 y un contenedor para el backend FastAPI. En la configuración base, el backend se ejecuta como un único proceso de Uvicorn.

A nivel de threads, el sistema define explícitamente un hilo de fondo llamado `ReservationCleaner`, encargado de revisar periódicamente las reservas vencidas y liberar los asientos expirados. Este hilo permite realizar tareas de mantenimiento sin bloquear el flujo principal de atención HTTP.

Además, el sistema utiliza mecanismos asincrónicos mediante `asyncio` para la gestión de WebSockets y para las pruebas concurrentes. El backend también utiliza un pool de conexiones a PostgreSQL, configurado para reutilizar conexiones y atender múltiples solicitudes sin abrir una conexión nueva por cada request.

Para medir el rendimiento se ejecutó una prueba de 200 solicitudes de reserva con un nivel de concurrencia de 50. El sistema completó la prueba en 0,601 segundos, con un throughput aproximado de 332,68 requests por segundo. Las 200 solicitudes fueron exitosas y no se registraron fallas en ese escenario.

Uso de CPU y memoria

El uso de CPU y memoria se midió durante la prueba de rendimiento mediante docker stats. En la corrida realizada, el contenedor del backend (ticket-api) alcanzó un pico observado de CPU de 124,01% y un consumo de memoria aproximado de 54 MiB. En Docker, el uso de CPU puede superar el 100% porque se calcula en relación con los núcleos disponibles; por lo tanto, ese valor representa aproximadamente el uso de 1,24 núcleos de CPU en ese instante.

Por su parte, el contenedor de PostgreSQL (ticket-db) alcanzó un pico observado de CPU de 26,16% y un consumo de memoria aproximado de 82 MiB. Estos valores muestran que, para el escenario probado, el sistema mantuvo un consumo de memoria bajo.

La latencia promedio observada fue de 0,1417 segundos, con una latencia máxima de 0,2286 segundos y una latencia p95 de 0,2259 segundos. Esto indica que el 95% de las solicitudes se respondió en aproximadamente 226 milisegundos o menos.

Cantidad de tokens utilizados

Usando la herramienta ccusage (npx ccusage@latest), se obtuvo el siguiente resumen para la cantidad de tokens utilizados entre refinamientos y prompt inicial:

Agent	Models	Input	Output	Cache Create	Cache Read	Total Tokens	Cost (USD)
All		679	99,831	241,740	5,714,582	6,056,832	\$2.82
- Claude	- haiku-4-5 - opus-4-7 - sonnet-4-6	679	99,831	241,740	5,714,582	6,056,832	\$2.82
All		33	7,865	216,741	485,540	710,179	\$1.79
- Claude	- opus-4-7	33	7,865	216,741	485,540	710,179	\$1.79
All		57	18,607	117,640	1,174,882	1,311,186	\$1.79
- Claude	- opus-4-7	57	18,607	117,640	1,174,882	1,311,186	\$1.79

Por día: 6.056.832 + 710.179 + 1.311.186

Total de tokens utilizados: 8.078.197 (+8M).

Cantidad de refinamientos

El desarrollo no se considera como una única generación directa, sino como un proceso iterativo. Luego de obtener una primera versión funcional, se revisó el código y se identificaron puntos de mejora relacionados con la concurrencia, la consistencia de datos y la posibilidad de ejecutar mediciones reales.

Se pueden identificar dos refinamientos principales. El primero estuvo enfocado en la correctitud concurrente: ordenar y deduplicar los asientos antes de procesar reservas múltiples, reforzar la confirmación de compra como operación transaccional completa, corregir el acceso a resultados de PostgreSQL y mejorar el batch de pruebas concurrentes para simular usuarios distintos.

El segundo refinamiento estuvo enfocado en la ejecución y medición del sistema: incorporación de Docker Compose, creación de una prueba de rendimiento, corrección del manejo del JWT, actualización de hashes del seed y ajuste de dependencias para ejecutar correctamente sobre Python 3.14.

Esta métrica es importante porque muestra que el uso de IA no reemplaza la revisión técnica humana. La herramienta puede generar una base funcional, pero sigue siendo necesario analizar, validar, ejecutar pruebas y ajustar el código para mejorar su correctitud.

Grado de correctitud del código en forma concurrente (sincronización adecuada, deadlocks, race condition, exclusión mutua)

El grado de correctitud concurrente se analizó observando cómo el sistema maneja sincronización, exclusión mutua, race conditions y posibles deadlocks.

La sincronización principal se apoya en PostgreSQL. La reserva de un asiento se realiza mediante una actualización condicional que solo tiene éxito si el asiento se encuentra disponible. Esto permite que, si dos usuarios intentan reservar el mismo asiento al mismo tiempo, solo una operación pueda modificarlo exitosamente.

Las race conditions no se evitan en el sentido de que pueden ocurrir intentos simultáneos sobre el mismo recurso, pero sí se controlan correctamente. En la prueba concurrente realizada, 10 usuarios intentaron reservar el mismo asiento al mismo tiempo. El resultado fue 1 reserva exitosa y 9 rechazos controlados, lo cual coincide con el comportamiento esperado.

Respecto a deadlocks, se identificó que las reservas múltiples podían presentar riesgo si distintos usuarios solicitaban los mismos asientos en diferente orden. Para reducir ese riesgo, se refinó la lógica ordenando los identificadores antes de procesar la reserva. De esta forma, las transacciones tienden a adquirir los recursos en una secuencia consistente.

La exclusión mutua sobre los asientos se delega principalmente en la base de datos, mientras que en la aplicación se utilizan mecanismos como `threading.Lock`, `threading.Event`, `threads` de fondo, `WebSockets` y `asyncio` para coordinar tareas auxiliares y actualizaciones en tiempo real.

Conclusiones

Este trabajo evaluó la capacidad de las herramientas de IA generativa para desarrollar sistemas de programación concurrente. El resultado fue positivo: se logró generar una arquitectura funcional que aborda correctamente los principales desafíos de sincronización en un sistema de venta de entradas bajo demanda concurrente.

Un hallazgo clave fue que Claude Code comprendió acertadamente dónde radica el verdadero problema de la concurrencia: no en la aplicación, sino en el control de acceso al recurso compartido. La solución implementada utilizó operaciones atómicas en base de datos, lo que se alinea con los principios de linearizabilidad descritos por Herlihy & Wing (1990). Las pruebas confirmaron que ante solicitudes simultáneas, solo una logra reservar el asiento, previniendo así las race conditions de forma efectiva.

Sin embargo, la versión inicial requirió refinamiento significativo. Se encontraron validaciones incompletas en compras múltiples y riesgos potenciales de deadlock. Este resultado respalda la observación de Boehm (2005) sobre que los mecanismos de concurrencia no pueden automatizarse completamente. La supervisión técnica humana resultó esencial para asegurar correctitud.

El rendimiento medido fue satisfactorio: 332.68 requests por segundo con latencia promedio de 0.1417 segundos. Además, la incorporación de logging automático de condiciones de carrera proporcionó visibilidad sobre conflictos que ocurrían en tiempo real, un aspecto crucial para diagnosticar problemas en producción.

Un aprendizaje importante fue reconocer que la IA acelera el desarrollo significativamente, pero no reemplaza los principios fundamentales de ingeniería de software. La validación, las pruebas bajo carga y la revisión técnica siguen siendo imprescindibles. De hecho, con código que se genera más rápido, estas prácticas se vuelven aún más críticas.

En conclusión, la generación de código para programación concurrente mediante herramientas de IA es viable y promisorio. Sin embargo, requiere supervisión constante, validación exhaustiva y aplicación rigurosa de principios de ingeniería de software. Para proyectos similares, se recomienda este enfoque siempre que se establezcan procesos de revisión y testing adecuados.

Referencias

Boehm, H. J. (2005). Threads cannot be implemented as a library. En Proceedings of the SIGPLAN Conference on Programming Language Design and Implementation, 40(6), 261-268. ACM.

Herlihy, M., & Wing, J. M. (1990). Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 12(3), 463-492. ACM.